File Path Traversal, Traversal Sequences Blocked with Absolute Path Bypass

Lab Information

Field	Value
Category	Path Traversal
Difficulty	Apprentice
Vulnerability	Path Traversal / Improper Input Validation
Impact	Arbitrary File Read

Summary

This lab demonstrates a Path Traversal vulnerability where the application attempts to prevent directory traversal by blocking traversal sequences such as `../`.

However, the application still accepts absolute file paths supplied by the user. By providing an absolute path directly, an attacker can bypass the implemented restriction and read sensitive files from the server filesystem.

Objective

Retrieve the contents of the sensitive file:

```
/etc/passwd
```

Attack Scenario

The application loads product images using a user-controlled `filename` parameter.

Unlike the previous lab, traversal sequences such as:

```
../
```

are filtered by the application.

However, the application does not properly validate whether the provided filename represents a safe file location. Since absolute paths are still accepted, an attacker can directly request sensitive files by providing their full filesystem path.

Steps to Reproduce

1. Identify the image loading functionality.

Example request:

```
GET /image?filename=7.jpg
```

1. Send the request to Burp Suite Repeater.

2. Attempting a traditional traversal payload:

```
filename=../../../../etc/passwd
```

is blocked by the application.

1. Bypass the restriction by providing an absolute path:

```
filename=/etc/passwd
```

1. Send the modified request.

2. Observe that the server returns the contents of `/etc/passwd`.

3. The lab is successfully solved.

Evidence

Normal Request

The application loads images through the filename parameter:

```
GET /image?filename=7.jpg
```

Blocked Traversal Attempt

Traditional traversal sequences are filtered:

```
filename=../../../../etc/passwd
```

Successful Bypass

An absolute path bypasses the restriction:

```
filename=/etc/passwd
```

Result

The server returns sensitive Linux system file contents:

```
root:x:0:0:root:/root:/bin/bash
```

Security Impact

Attackers can bypass insufficient file path validation and read arbitrary files from the server filesystem.

Sensitive files may contain:

- Application configuration
- Database credentials
- API keys
- Source code
- User information
- Operating system details

This information can be used for further attacks against the application or underlying infrastructure.

Root Cause

The application attempted to mitigate Path Traversal by blocking specific traversal sequences but failed to properly validate the final file path.

The security control focused only on blocking a specific payload pattern (`../`) rather than enforcing that requested files remained inside the intended directory.

Remediation

- Avoid accepting raw filesystem paths from users.
- Use an allowlist of permitted files instead of user-controlled paths.
- Resolve the final path before accessing the file and verify that it remains inside the allowed directory.
- Do not rely on blacklist-based filtering for security controls.
- Reject absolute paths when they are not required.

Pentester's Notes

When a known attack pattern is blocked, the next step is to analyze the security control itself.

In this case, the application blocked:

```
../
```

but did not prevent:

```
/etc/passwd
```

This demonstrates why blacklist-based defenses are often ineffective. Security testing should focus on the underlying logic rather than only testing a single payload.

Key Takeaways

- Blocking `../` alone does not prevent Path Traversal.
- Absolute paths can bypass weak traversal filters.
- Input validation should verify the final resolved path, not just search for malicious strings.
- Effective security controls should use allowlists and proper authorization logic instead of simple blacklists.